

Telecom Customer Churn Prediction

A Data Analysis and Machine Learning Project

Prepared by: Mahesh Thakare

Tools used: Python, Pandas, Scikit-learn, XGBoost, Matplotlib, Seaborn, Streamlit

1. Objective

Telecom companies lose a significant number of customers every year, and it costs a lot more to acquire a new customer than to retain an existing one. The goal of this project was to first understand why customers were leaving using exploratory data analysis, and then build a machine learning model that can predict which existing customers are likely to churn, so the business can act before they leave.

2. Dataset

Dataset used: IBM Telco Customer Churn dataset (from Kaggle)

Records: 7,043 customers

Features: 21, including demographics, account information, contract type, billing details, and the services each customer had subscribed to.

3. Exploratory Data Analysis

Before touching any model, I spent time cleaning the data (TotalCharges had blank values for new customers, which I filled with 0) and exploring it to understand what was actually driving churn. The overall churn rate in the dataset was 26.5%. A few patterns stood out clearly:

- Contract type mattered the most. Month-to-month customers churned at 42%, while two-year contract customers churned at only 3%.
- Customers paying by electronic check churned at 45%, almost three times higher than customers on automatic payment methods (15%).
- New customers were the most likely to leave. Churn was over 54% in the first 6 months, and dropped to around 15% after 3-4 years.
- Fiber Optic customers churned more (30%) than DSL customers (20%), even though it's the premium service. This was mostly driven by fiber customers who did not have Tech Support or Online Security add-ons.

These findings weren't just interesting on their own, they directly shaped how I engineered features for the ML model later, since I made sure tenure, contract type, and service add-ons were represented properly instead of just throwing raw columns at a model.

4. Machine Learning Approach

Since only about a quarter of customers actually churn, accuracy is a misleading metric here — a model that just predicts "No Churn" for everyone would already score around 73% accuracy while being completely useless. So I focused on Recall instead, since for this business problem, missing an actual churner is worse than a false alarm.

I trained and compared three models: Logistic Regression (as a simple, interpretable baseline), Random Forest, and XGBoost.

Model	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.515	0.797	0.625	0.844
Random Forest	0.546	0.730	0.625	0.841
XGBoost (untuned)	0.540	0.671	0.598	0.824
XGBoost (tuned) — final model	0.490	0.810	0.610	0.840

The interesting part was that the untuned XGBoost model actually performed the worst of the three, even though it's usually considered the more powerful algorithm. Instead of accepting that, I ran RandomizedSearchCV to tune XGBoost's hyperparameters, optimizing specifically for Recall. After tuning, XGBoost improved to 81% Recall and a ROC-AUC of 0.84, overtaking the baseline models.

This was probably the biggest learning from the project: a more complex model doesn't automatically mean better results. It only became the best model once it was properly tuned, and comparing an untuned complex model against a simple baseline isn't really a fair comparison.

5. What the Model Learned

I checked the final model's feature importances to see whether it had picked up on the same patterns as the EDA. It had. Contract type, internet service type, tenure, and whether a customer had Tech Support or Online Security were consistently the top predictors, which matched what the earlier analysis had already shown. This was a good sanity check that the model wasn't just fitting noise.

6. Turning Predictions into Business Action

A churn probability score by itself isn't very useful to a customer success team, so I converted each customer's predicted probability into one of four risk categories:

- Low Risk: 0-30% probability
- Medium Risk: 30-50% probability
- High Risk: 50-70% probability
- Critical Risk: 70-100% probability

This makes the output something a non-technical team could actually use to prioritize who to reach out to first.

7. Deployment

I deployed the final model as a Streamlit web app. A user can enter a customer's account, billing, and service details, and the app returns a churn probability, a risk category, and a retention recommendation for high-risk customers. The app rebuilds the exact same cleaning and feature engineering steps that were used during training, so predictions on new customer data stay consistent with how the model was trained.

8. Conclusion

This project took the analysis beyond just explaining past churn, and turned it into something that can be used going forward: a model that scores current customers, and an app that makes that score usable by a non-technical team. The main takeaways for me were choosing the right metric for the actual business problem instead of defaulting to accuracy, and being careful not to assume a complex model is better without validating it through tuning.